上位机车辆状态显示

1. 外设启动流程

当前程序，启动顺序为：

```
执行 docker/demo.sh
  ↓
创建容器、加载 ROS 2 环境
  ↓
启动 docker_startup.launch.py
  ├─ WebSocket 通信服务
  ├─ RTK 串口驱动
  ├─ RTK 坐标与航向转换
  ├─ 激光雷达驱动
  ├─ 超声波驱动
  └─ CAN 反馈节点
  ↓
检查外设是否正常
  ├─ RTK 定位有效
  ├─ RTK 航向有效
  ├─ 雷达点云正常
  ├─ 超声波数据正常
  └─ CAN 反馈有效
  ↓
全部通过 → 启动 demo.launch.py
  ↓
循迹、CAN 执行、围栏等控制节点就绪
  ↓
设备状态：就绪
车辆状态：循迹中 / 遥控模式
```

因此上位机同时显示各项状态，哪一项先正常就先显示“就绪”。

2. 设备总状态

设备总状态采用五种：

状态值	上位机文字	判断方式
`waiting_connection`	等待连接	上位机打开后，尚未收到 Jetson 状态
`disconnected`	断线	曾经连接，但状态通信断开或超时

状态值	上位机文字	判断方式
`starting`	启动中	已连接，正在启动外设、检查或启动控制程序
`startup_failed`	启动失败	必需外设检查失败，或控制程序启动失败
`ready`	就绪	外设检查通过，控制程序已可用

其中，**等待连接、断线由上位机判断**，因为 Jetson 无法通过已经断开的连接报告自己断线。

正常转换：

```
等待连接 → 启动中 → 就绪
                ↘ 启动失败
```

任何已连接状态都可能转为“断线”；
重新连接后，直接采用 Jetson 最新上报状态。
上位机连接较晚时，不重放整个启动过程，直接展示当前结果。

3. 外设状态

显示四个外设状态：

外设	“就绪”条件	需要区分的错误
RTK	定位、航向数据均有效且持续更新	串口打开失败、无定位、无有效航向、数据超时
激光雷达	驱动运行，持续收到有效点云	驱动启动失败、无点云、点云超时
超声波	串口正常，持续收到四路数据	串口打开失败、数据不足四路、数据超时
CAN/VCU	CAN 接口正常，收到有效车辆反馈	接口打开失败、无反馈、反馈无效、发送失败

每个外设只需要三种内部状态：

状态值	显示
`starting`	启动中 / 等待数据
`ready`	就绪
`error`	对应错误文字

例如：

```
设备：启动失败
```

```
RTK          就绪
激光雷达     超过 5 秒未收到点云
超声波       就绪
CAN/VCU      就绪
```

错误信息要反映已确认的现象。比如只能确认“未收到点云”时，就显示这个结果；不能直接断言“雷达未上电”。

路径、围栏文件和控制节点不必增加外设卡片；它们启动失败时，直接作为设备“启动失败”的原因显示，例如“路径文件解析失败”。

4. 启动后的车辆状态

车辆状态只在设备“就绪”且控制状态有效时显示：

状态值	显示	判断依据
`trajectory`	循迹中	执行节点实际采用循迹指令
`remote`	遥控模式	执行节点实际采用来自 9010 的遥控指令
`null`	暂不显示	尚未就绪、故障停止或状态未知

5. 供给上位机的数据

用一个完整 JSON 同时提供这三部分状态，每秒发送一次，状态变化时立即发送。
上位机连接后立即收到最新完整数据。

可以新增独立状态接口：

```
ws://JetsonIP:9011/status
```

数据结构如下，示例表示启动失败：

```
{
  "type": "device_status",
  "version": 1,
  "boot_id": "boot-001",
  "seq": 25,
  "device_state": "startup_failed",
  "device_message": "激光雷达检查失败",
  "vehicle_state": null,
  "peripherals": {
    "rtk": {
      "state": "ready",
```

```
      "errors": []
    },
    "lidar": {
      "state": "error",
      "errors": [
        {
          "code": "LIDAR_NO_DATA",
          "message": "启动检查超时，未收到点云数据"
        }
      ]
    },
    "ultrasonic": {
      "state": "ready",
      "errors": []
    },
    "can": {
      "state": "ready",
      "errors": []
    }
  }
}
```

字段约定：

字段	用途
`type`、`version`	消息类型及协议版本
`boot_id`	区分每次启动，防止沿用上次启动状态
`seq`	递增序号，识别消息顺序
`device_state`	`starting`、`startup_failed`、`ready`
`device_message`	总体说明或失败原因，正常时可为空
`vehicle_state`	`trajectory`、`remote` 或 `null`
`peripherals`	每个外设的状态及错误列表
`errors[].code`	稳定错误码，供上位机程序判断
`errors[].message`	可直接显示的中文错误说明

正常进入循迹时：

```
{
  "device_state": "ready",
  "device_message": "",
```

```
  "vehicle_state": "trajectory"
}
```

这里仅展示变化字段；实际传输仍发送完整结构。
进入 9010 遥控后，`vehicle_state` 改为 `remote`。